SAHAYAK: SMART MUNICIPAL COMPLAINT MANAGEMENT SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Bhavishy Jain (EN23IT301036)
Sahil Mandloi (EN23CS304062)
Yash Arora (EN23CS304088)

Under the Guidance of

Prof. Sandeep Veerwani
Prof. Hariom Patidar

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

Jan-June 2026

SAHAYAK: SMART MUNICIPAL COMPLAINT MANAGEMENT SYSTEM

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Bhavishy Jain (EN23IT301036)
Sahil Mandloi (EN23CS304062)
Yash Arora (EN23CS304088)

Under the Guidance of

Prof. Sandeep Veerwani
Prof. Hariom Patidar

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

Jan-June 2026

Report Approval

The project work **"Sahayak: Smart Municipal Complaint Management System" is** hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner
Prof. Sandeep Veerwani
Assistant Professor
Medicaps University

Internal Examiner
Prof. Hariom Patidar
Assistant Professor
Medicaps University

External Examiner
Name:
Designation
Affiliation

Declaration

I/We hereby declare that the project entitled **"Sahayak: Smart Municipal Complaint Management System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology in 'Computer Science Engineering' completed under the supervision of **Prof. Sandeep Veerwani & Prof. Hariom Patidar** Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

1. **Bhavishy Jain (EN23IT301036)**

2. **Sahil Mandloi (EN23CS304062)**

3. **Yash Arora (EN23CS304088)**

Signature and name of the student(s) with date

Certificate

I/We, **Prof. Sandeep Veerwani & Prof. Hariom Patidar** certify that the project entitled **"Sahayak: Smart Municipal Complaint Management System"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Bhavishy Jain, Sahil Mandloi, Yash Arora** is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

_____

Prof. Sandeep Veerwani & Prof. Hariom Patidar

Faculty of Enginnering

Medi-Caps University, Indore

_____

<Name of External Guide (If any)>

<Name of the Department>

<Name of the Organization

_____

Dr. Kailash C Bandhu
Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal,** who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik,** Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya,** Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash C Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Bhavishy Jain, Sahil Mandloi, Yash Arora
B.Tech. III Year (AI-B)
Department of Computer Science & Engineering
Faculty of Engineering
Medi-Caps University, Indore

Abstract

Urban civic services often struggle with fragmented complaint handling, poor visibility into issue resolution, delayed coordination between departments, and limited feedback loops with citizens. To address these challenges, this project presents **Sahayak**, a smart municipal complaint management system designed as a multi-role digital platform for citizens, municipal workers, Heads of Department, and administrators. The system enables end-to-end complaint registration, workflow supervision, field execution, feedback collection, notification delivery, and analytics-driven governance.

Sahayak has been implemented using a Node.js and Express backend, MongoDB for persistence, and an Expo React Native mobile application for role-based user access. The solution supports complaint creation with textual descriptions, image evidence, and geographic coordinates. Once a complaint is submitted, the backend enriches it with AI-assisted analysis such as probable department, urgency, and priority signals. Heads of Department can review queues, apply AI suggestions, assign one or multiple workers, monitor departmental status, and take workflow actions such as approval, rework, or cancellation. Workers receive assigned complaints, update progress, upload completion photos, and submit work for approval. Citizens can track complaint history, interact through complaint chat, view heatmaps of nearby complaints, upvote issues affecting their neighborhood, and provide satisfaction feedback after resolution.

The system also includes push notifications, real-time complaint messaging, recurring report scheduling, role-aware analytics, soft-delete and recovery flows for complaints, invitation management, and mobile-first complaint submission with responsive client-side flows and cached data access. These capabilities make the platform suitable not only for citizen interaction but also for internal operational management within civic departments.

The project demonstrates how a modern full-stack architecture can improve transparency, accountability, and coordination in urban grievance redressal. By combining workflow automation, mobile accessibility, analytics, and AI-supported routing, Sahayak offers a practical foundation for smarter municipal service delivery and scalable civic operations.

Keywords: Municipal complaint management, grievance redressal, civic-tech, React Native, Node.js, MongoDB, workflow automation, AI-assisted triage, notification system.

Table of Contents

	3.3.6 Sequence diagram	
	3.4 **Database Design**	
	3.4.1 Logical Database Design	
	3.4.2 Physical Database Design	
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	
	4.2 Testing Techniques and Test Plans (According to project)	
	4.3 Installation Instructions	
	4.4 End User Instructions	
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	
	5.2 Brief Description of Various Modules of the system	
	5.3 Snapshots of system with brief detail of each	
	5.4 Back Ends Representation (Database to be used)	
	5.5 Snapshots of Database Tables with brief description	
Chapter 6	Summary and Conclusions	
Chapter 7	Future scope	
	Appendix	
	Bibliography	
	List of Publications (If any)	
	Reprints of publications(If any)	
Note	**Results and Discussions may not be a separate chapter it may be included in main chapter**	

List of Figures

List of Tables

Abbreviations

Abbreviations	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DB	Database
Expo	Framework for React Native application development
HOD	Head of Department
JWT	JSON Web Token
PDF	Portable Document Format
SLA	Service Level Agreement
UI	User Interface
UX	User Experience
WS	WebSocket

Chapter 1 Introduction

1.1 Introduction

Sahayak is a smart municipal complaint management system developed to digitize the reporting, handling, tracking, and resolution of civic complaints. The project addresses the common limitations of manual or fragmented complaint systems by creating a unified platform where citizens, municipal workers, Heads of Department, and administrators can interact within a structured digital workflow. The project is built using a Node.js and Express backend, MongoDB database, and an Expo React Native mobile client. It supports complaint submission with images and location, workflow assignment, analytics, assistant support, notifications, reporting, and role-based operations.

1.2 Literature Review

Existing digital grievance systems and service management platforms suggest that effective civic issue resolution depends on four major qualities: easy complaint registration, clear role responsibility, traceable status progression, and feedback-supported closure. Many modern public service systems now integrate geolocation, media evidence, dashboards, and notifications to improve service accountability. Research and practical implementations in civic-tech also emphasize the importance of mobile accessibility, data-driven decision making, and operational transparency.

Sahayak aligns with these principles by not only allowing citizen complaints but also integrating worker workflows, HOD supervision, admin monitoring, complaint heatmaps, AI suggestion review, and scheduled reporting. Compared to a simple ticketing tool, the project reflects a broader municipal operations platform.

Most existing complaint systems focus either on complaint intake or on backend processing, but not both in an integrated way. In many implementations, citizens can register issues online, yet operational visibility for municipal staff remains weak. Likewise, internal workflow tools often lack public transparency, media-based evidence capture, or citizen feedback loops. Sahayak attempts to bridge this gap by combining citizen usability with operational supervision, analytics, and reporting.

In addition to generic grievance portals, recent civic-tech systems increasingly combine media evidence, map-based complaint visualization, role-based dashboards, and status transparency. Sahayak extends this idea by integrating citizen-side reporting with operational workflows for workers, Heads of Department, and administrators. This makes the system closer to a complete municipal operations platform than a simple complaint submission portal.

1.3 Objectives

- To provide a digital platform for structured municipal complaint registration.
- To support a multi-role workflow across citizen, worker, HOD, and admin actors.
- To enable complaint status tracking, evidence upload, and realtime complaint chat.
- To improve complaint routing using AI-assisted suggestions and department-level supervision.
- To provide analytics, heatmaps, notifications, and scheduled reports for governance support.

1.4 Significance

The project is significant because it addresses a practical public service problem rather than a purely academic example. It improves citizen visibility into municipal action, strengthens accountability through timeline-based workflow records, and helps operational staff coordinate work more efficiently. For academic learning, the project is also significant because it combines frontend development, backend APIs, database design, authentication, notifications, reporting, and software architecture in one integrated system.

The significance of this project lies not only in solving a civic problem but also in demonstrating how software engineering principles can be applied to public-service systems. The project integrates authentication, role-based authorization, mobile UX, backend modularity, analytics, reporting, and AI-assisted guidance into a single application. Thus, it has both social relevance and technical learning value.

1.5 Research Design

The project follows an applied software engineering design approach. First, the problem domain was studied in terms of stakeholders and complaint lifecycle. Next, the required modules were identified and mapped into user roles and service flows. A modular system architecture was then designed using separate controllers, services, routes, and mobile UI screens. The application was developed iteratively with focus on complaint workflows, followed by analytics, reporting, notifications, and support features.

1.6 Source of Data

The report content is based primarily on the actual codebase and repository structure of the Sahayak project. This includes backend route files, controllers, services, models, mobile screen flows, hooks, seeded demo accounts, and internal project documentation. For implementation understanding, the main sources are the project README, route definitions, package configuration files, and current app modules. The implementation also followed a repository-driven refinement process in which route design, model design, service decomposition, and role policies were iteratively aligned with the complaint lifecycle.

1.7 Chapter Scheme

Chapter 1 introduces the project background, objectives, and significance. Chapter 2 describes requirements specification. Chapter 3 presents design aspects including algorithmic flow, function orientation, diagrams, and database design. Chapter 4 explains implementation, testing, and maintenance aspects. Chapter 5 discusses results with module representation, interface details, and database views. Chapter 6 provides the summary and conclusion. Chapter 7 outlines future scope. The report ends with appendix and bibliography.

Chapter 2. Requirement Specification

2.1 User Characteristics

Stakeholder	Primary Responsibilities
Citizen	Register, log in, submit complaints, upload proof images, track status, chat, upvote nearby issues, manage notifications, provide feedback, and vote on satisfaction.
Municipal Worker	View assignments, start work, update complaint status, upload completion photos, submit work for approval, chat on complaint threads, and review personal analytics.
Head of Department	Review queues, assign one or more workers, manage workers and invitations, approve completion, send for rework, review AI suggestions, and generate reports.
Admin	Manage users and departments, review deleted complaints, process special requests, manage broadcast notifications, supervise festival events, and monitor platform-level operations.

2.2 Functional Requirements

ID	Requirement
FR-1	The system shall support secure registration, login, logout, token refresh, email verification, and password reset.
FR-2	The citizen shall be able to create a complaint with text, media, and location data.
FR-3	The system shall store complaint status history and display complaint detail.
FR-4	The citizen shall be able to track complaint status, chat on complaint threads, and receive notifications.

FR-5	The worker shall be able to view assigned complaints and update work progress.
FR-6	The worker shall be able to upload completion photos and submit work for approval.
FR-7	The HOD shall be able to review complaints and assign one or more workers.
FR-8	The HOD shall be able to approve completion, request rework, cancel complaints, and manage worker tasks.
FR-9	The system shall provide analytics dashboards, heatmaps, and reports.
FR-10	The admin shall be able to manage users and deleted complaint recovery flows.
FR-11	The system shall support AI-assisted complaint review and multilingual assistant support.
FR-12	The system shall support scheduled and email-based reports for HOD and admin roles.
FR-13	The system shall support complaint messaging between citizen, worker, HOD, and admin users on complaint-specific threads.
FR-14	The system shall support soft-delete, restore, and permanent purge workflows for complaints under admin control.
FR-15	The system shall support worker invitation workflows for HOD users and acceptance tracking in the backend.
FR-16	The system shall support admin-targeted broadcast notifications and festival-event-based operational alerts.

2.3 Dependencies

The project depends on Node.js runtime, Express, MongoDB, Mongoose, Expo, React Native, TanStack Query, Expo Notifications, Cloudinary for media storage, Resend or configured email infrastructure for verification and report delivery, WebSocket support for realtime updates, and AI provider integration for assistant and complaint analysis workflows. Some features such as push delivery, email verification, speech-based assistant support, and AI-assisted analysis require correct third-party service configuration through environment variables.

Some modules depend on third-party service configuration. For example, image upload workflows depend on Cloudinary configuration, email verification and password reset depend on email service credentials, push notifications depend on Expo notification registration, and AI-assisted assistant features depend on configured AI provider keys. Therefore, full project behavior assumes proper backend environment variable setup.

2.4 Performance Requirements

The system should provide responsive complaint list loading, efficient complaint status updates, acceptable image upload performance over standard mobile networks, and reliable realtime message propagation. The backend should handle concurrent authenticated requests and support reporting jobs without affecting core complaint lifecycle operations. Query caching and pagination are used in the mobile app to improve responsiveness.

The complaint list and detail views should support pagination and partial caching to avoid loading excessive data in one request. Report generation should be handled in a controlled way so that large exports do not block complaint operations. Realtime complaint chat should provide acceptable responsiveness under normal mobile network conditions.

2.5 Hardware Requirements

Component	Requirement
Development Machine	System capable of running Node.js, MongoDB, code editor, and Expo development tools
Server Environment	Any environment capable of running Node.js backend and MongoDB connection
Mobile Device	Android smartphone with camera, location permission, and internet access for full usage
Network	Internet connection required for live APIs, notifications, and realtime features

2.6 Constraints & Assumptions

The current project assumes valid backend configuration, user role provisioning, and availability of internet connectivity for most operations. Some admin features remain more backend-complete than mobile-complete. AI suggestions are advisory and require human supervisory judgement. The system is designed as a strong practical project and not yet a full government-integrated production deployment.

Chapter 3. System Design and Architecture

3.1 Algorithm (if Applicable)

The central algorithmic logic of Sahayak lies in complaint lifecycle progression. At a high level, the algorithm follows this sequence: authenticate user, validate complaint input, store complaint data, generate AI hints if enabled, notify relevant audiences, place complaint into departmental queue, assign workers through HOD workflow, accept worker updates, collect completion evidence, route complaint to approval stage, close complaint on approval, and finally allow citizen feedback and satisfaction voting.

The implemented complaint workflow is state-driven. In the present backend, the primary complaint states are pending, assigned, in-progress, pending-approval, needs-rework, resolved, and cancelled. Status transitions are role-restricted and validated through workflow services and authorization policies.

3.2 Function Oriented Design for procedural approach

From a function-oriented perspective, the system can be decomposed into major processing functions: authentication, complaint creation, complaint query, worker assignment, workflow transition, feedback management, notification delivery, analytics generation, report generation, and administration. Each function accepts structured inputs, performs validation and processing, and produces a defined output. This style is visible in the route-controller-service pattern used in the backend, where each request flows through a predictable chain of processing.

3.3 System Design

The system design is based on a layered full-stack architecture. The mobile client acts as the presentation layer. The Express API acts as the application layer. Services implement workflow and domain logic. MongoDB collections act as the persistent data layer. External services support media storage, email, notifications, and AI features.

3.3.1 Data Flow Diagrams (Level 0,Level1)

In Level 0 DFD, Sahayak can be treated as a single complaint management system receiving inputs from Citizen, Worker, HOD, and Admin actors, and exchanging complaint, status, report, and notification data with them. In Level 1 DFD, the system can be broken into sub-processes such as authentication, complaint registration, complaint assignment, worker update, notification handling, analytics/report generation, and admin management.

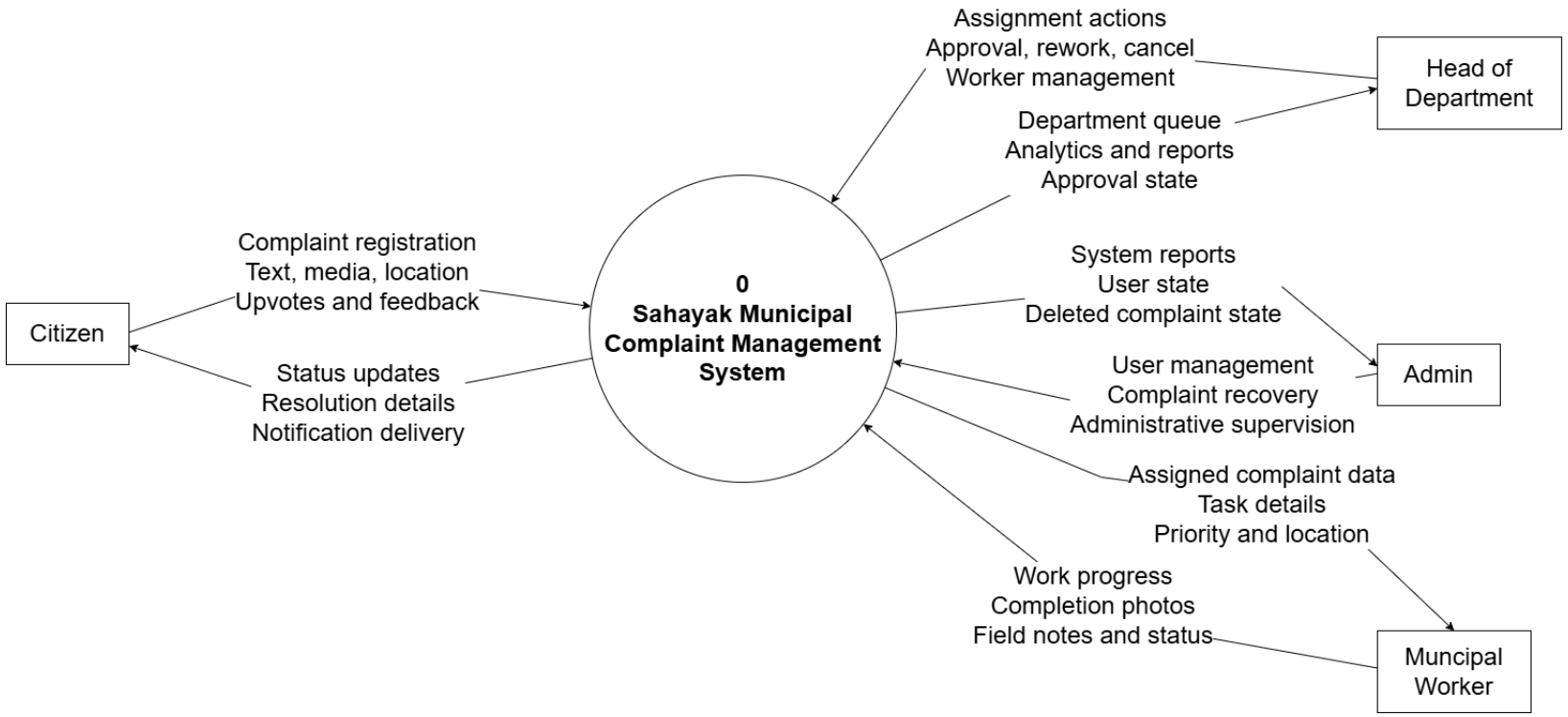


Figure 3.3.1: Level 0 DFD of Sahayak showing Citizen, Worker, HOD, Admin, and the central complaint management process.

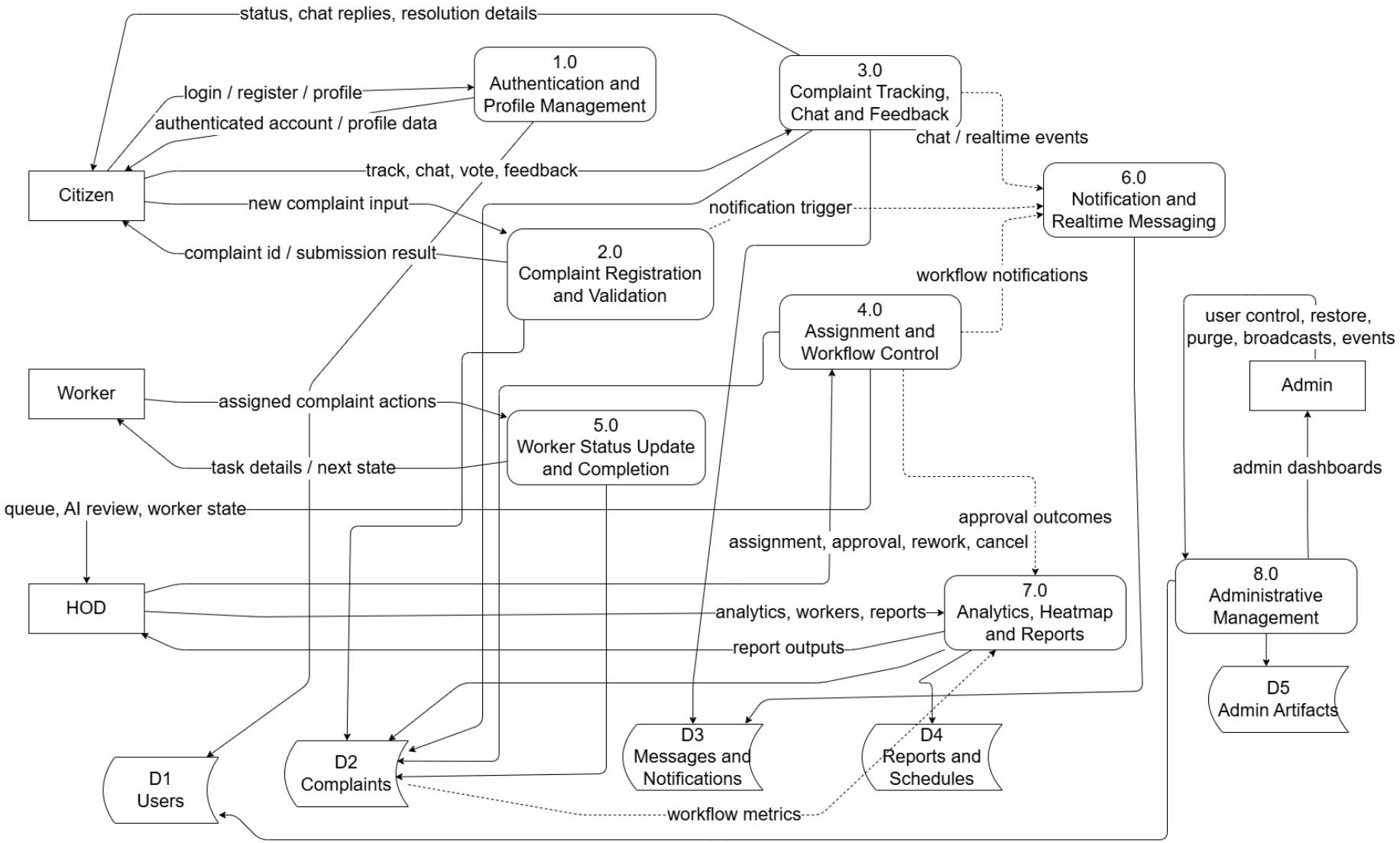


Figure 3.3.2: Level 1 DFD showing authentication, complaint registration, assignment, status update, notification, analytics, and report generation processes.

3.3.2 Activity Diagram

The activity diagram should capture the sequence starting from citizen complaint submission, validation, AI analysis, HOD review, worker assignment, field status updates, completion upload, HOD approval, and citizen feedback. This diagram will best show the decision points and role transitions in the workflow.

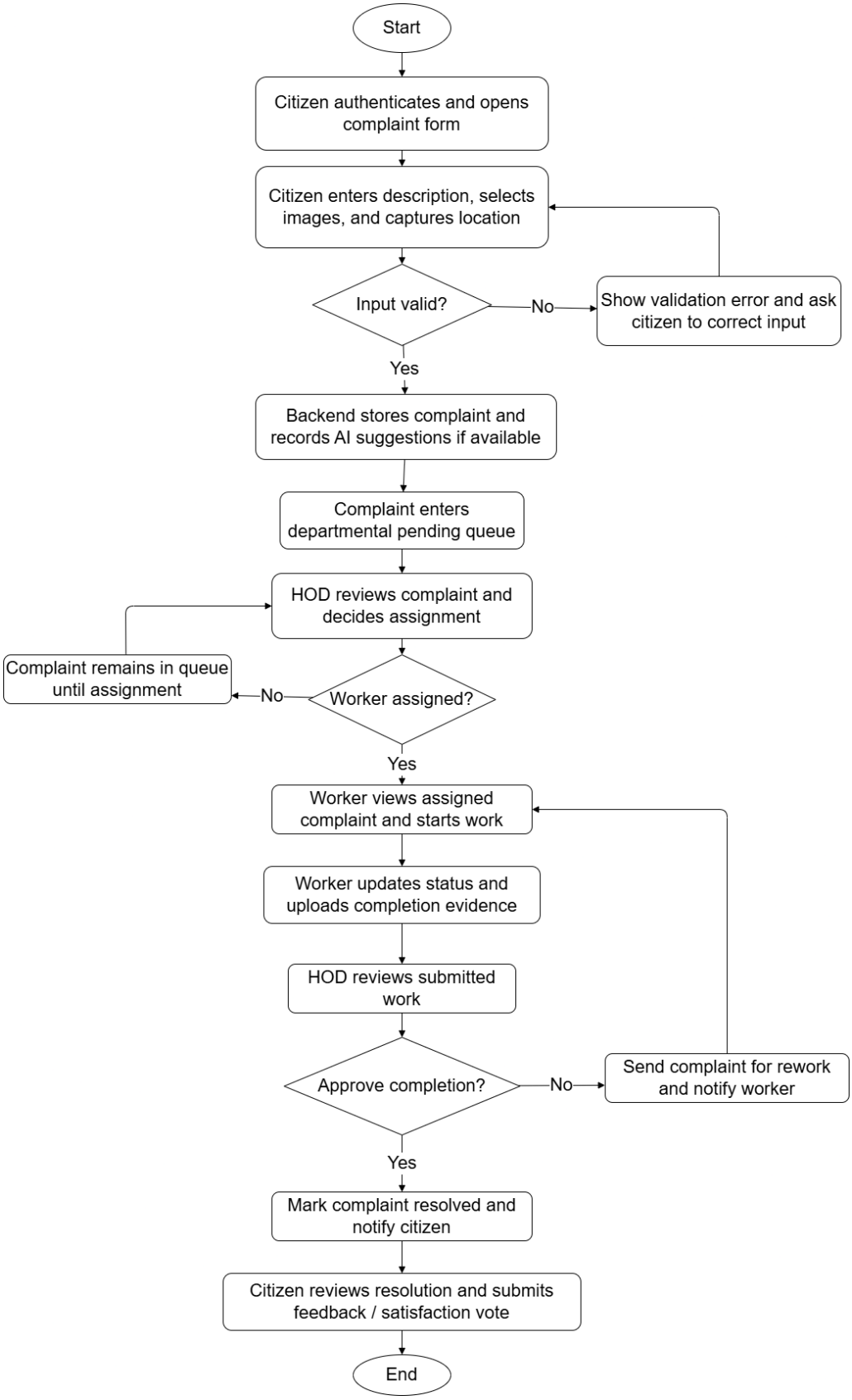


Figure 3.3.3: Activity diagram representing complaint flow from registration to final citizen feedback.

3.3.3 Flow Chart

The flow chart for Sahayak should represent the procedural execution path of complaint handling. It may start from login or register, continue to complaint submission, verify department and evidence, check assignment and status transitions, and terminate at either complaint resolution or rework loop.

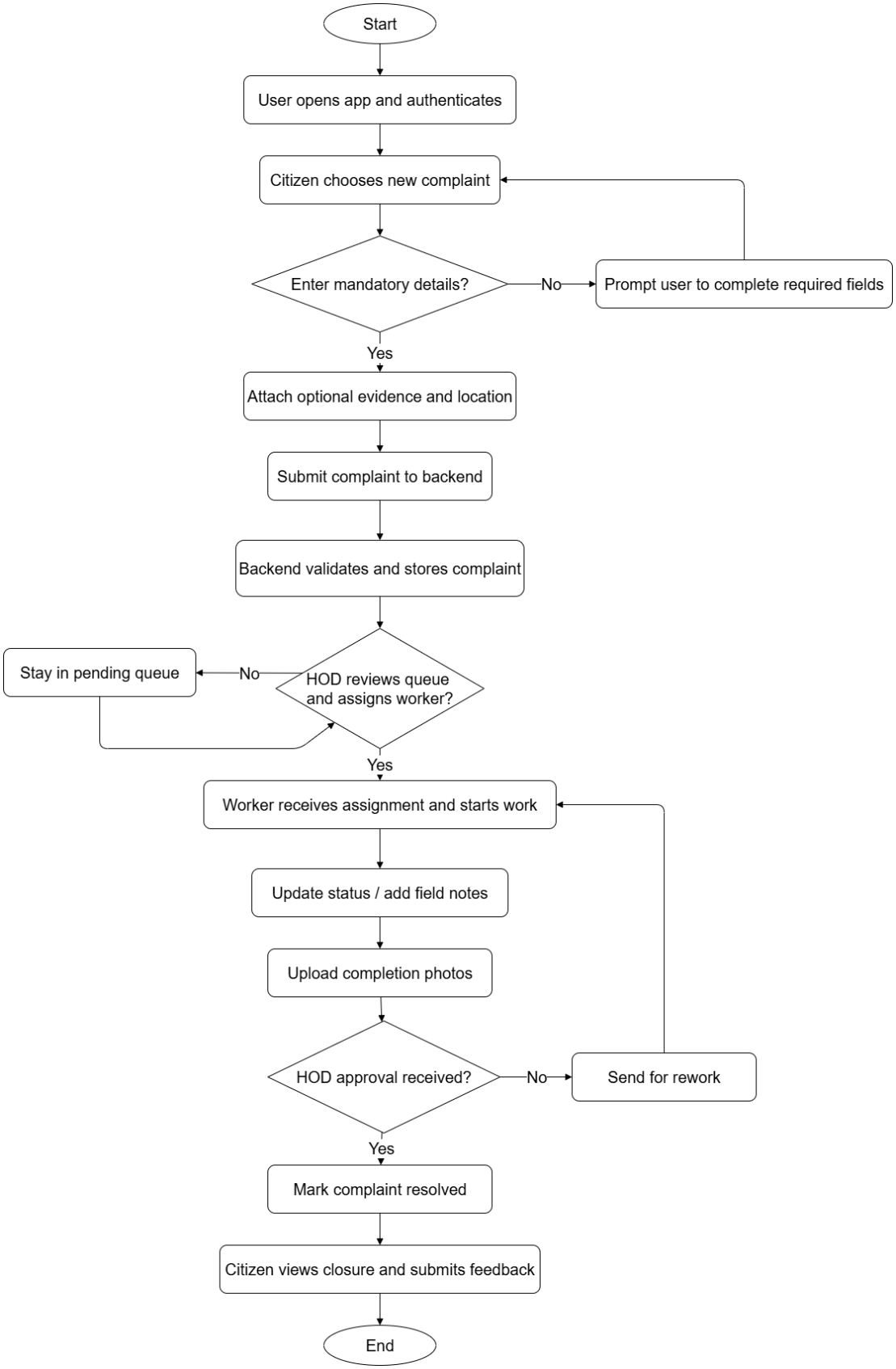


Figure 3.3.4: Flow chart of complaint registration, review, assignment, execution, approval, and closure.

3.3.4 Class Diagram

The class diagram should represent the actual core backend classes and role-specialized domain actors used in the project. The main superclass is User, from which role-oriented actors such as Citizen, Worker, HeadOfDepartment, and Admin can be conceptually derived for design explanation. Other major classes include Complaint, ComplaintAssignment, ComplaintHistory, ComplaintFeedback, ComplaintMessage, Notification, WorkerInvitation, ReportSchedule, ComplaintSpecialRequest, FestivalEvent, and AdminNotificationBroadcast. Complaint is the central operational class and aggregates assignment, history, feedback, and AI-analysis structures.

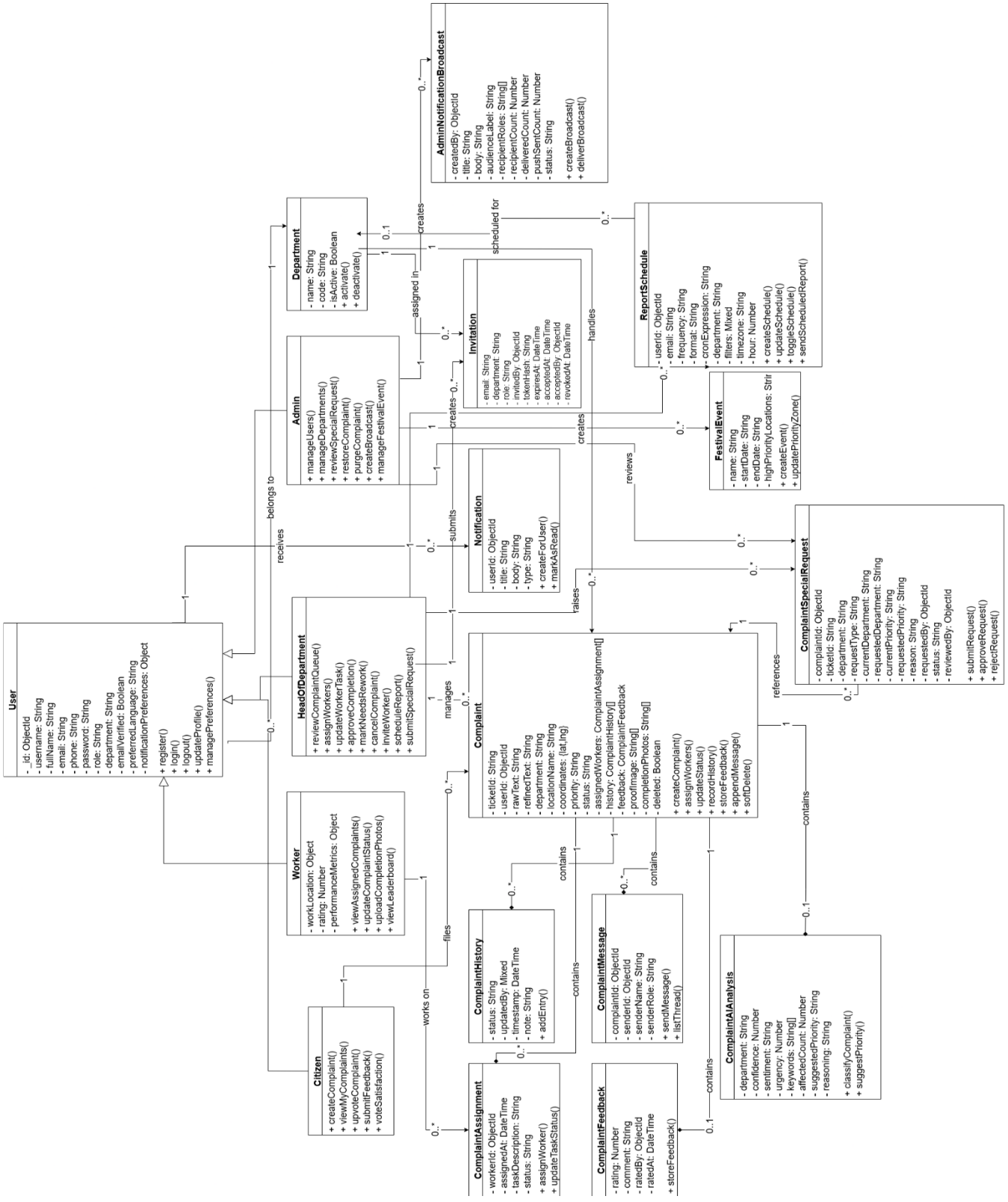


Figure 3.3.5: UML class diagram of core Sahayak actors, complaint domain classes, and support services.

3.3.5 ER Diagram

The ER diagram should show the logical data relationships among the core MongoDB entities. The most important entities are User, Complaint, Department, Notification, WorkerInvitation, ReportSchedule, ComplaintSpecialRequest, ComplaintMessage, FestivalEvent, and AdminNotificationBroadcast. Complaint is the central entity and logically relates to user ownership, department context, worker assignment, workflow history, feedback, and complaint-specific messaging.

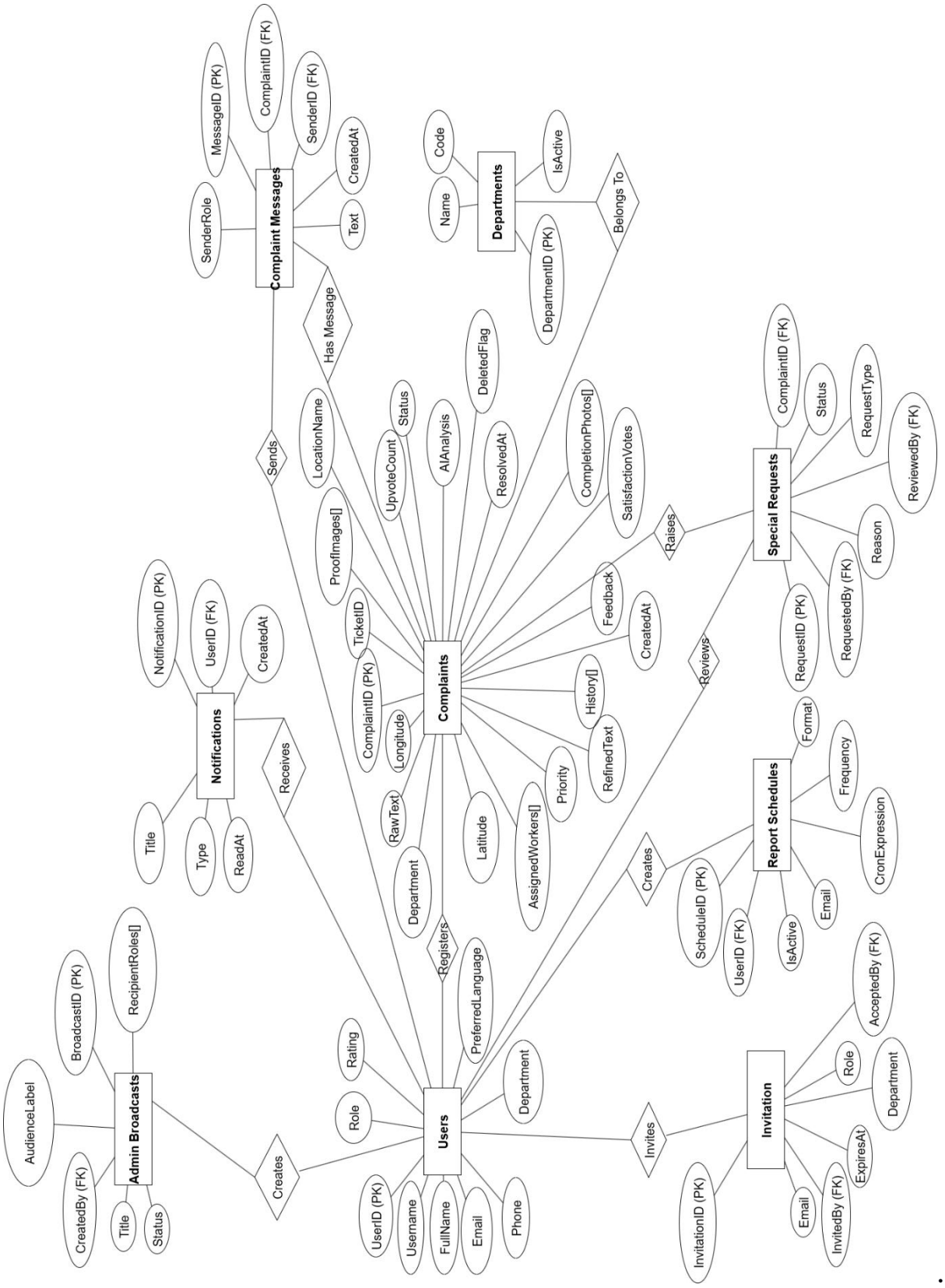


Figure 3.3.6: ER diagram showing logical relationships among core Sahayak data entities.

.

3.3.6 Sequence diagram

A sequence diagram is especially useful for depicting complaint submission and assignment. The actors in the sequence include Citizen App, Backend API, Database, HOD User, Worker User, and Notification Service. The sequence may show complaint registration, persistence, AI enrichment, queue listing, assignment, worker updates, and closure notifications.

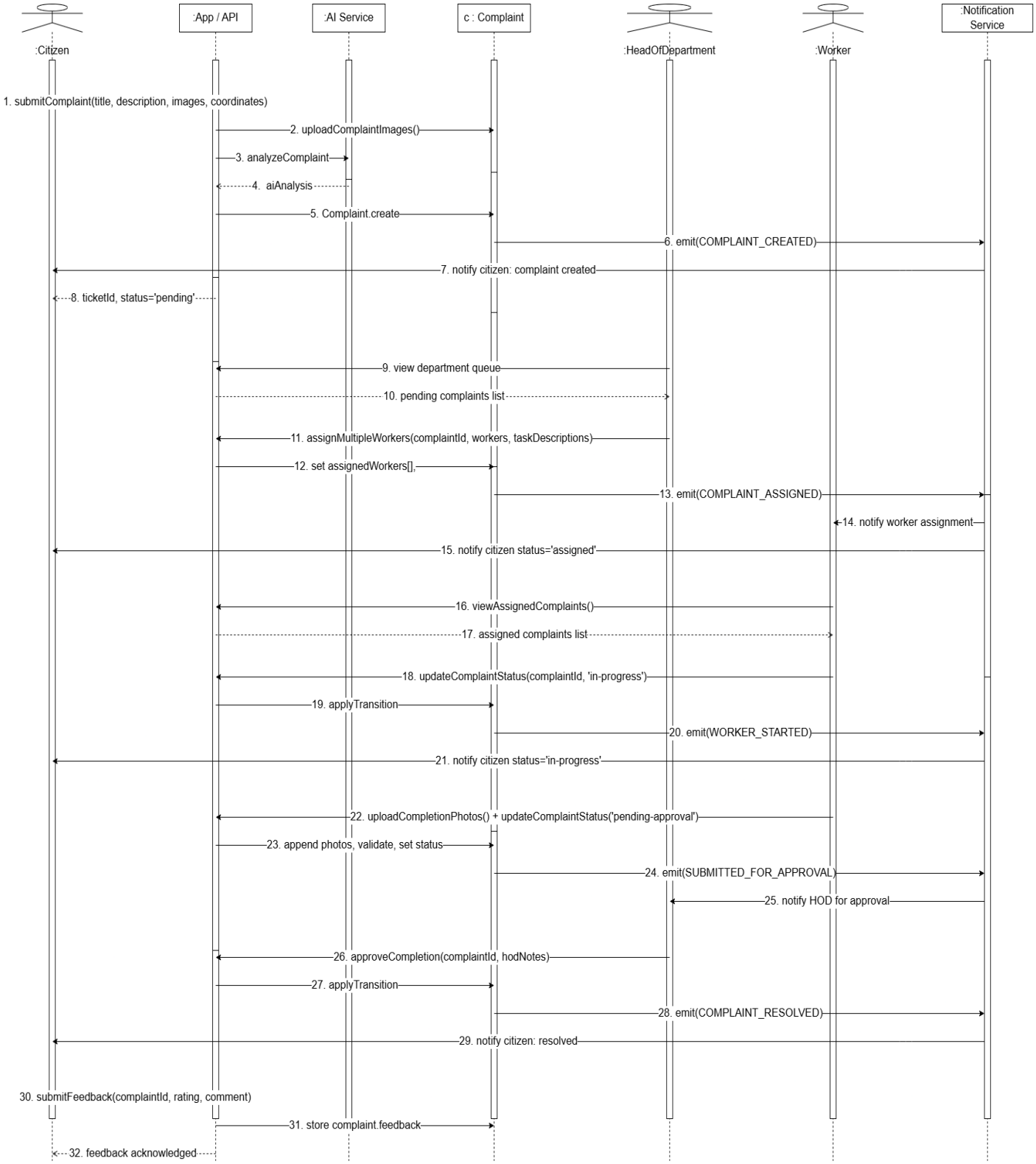


Figure 3.3.7: Sequence diagram of citizen complaint submission, HOD assignment, worker progress update, and complaint resolution.

3.4 Database Design

The project uses MongoDB for persistent storage. The data model is document-oriented, which suits the complaint structure because complaints naturally include history, media, assignments, votes, feedback, and other embedded or related details.

3.4.1 Logical Database Design

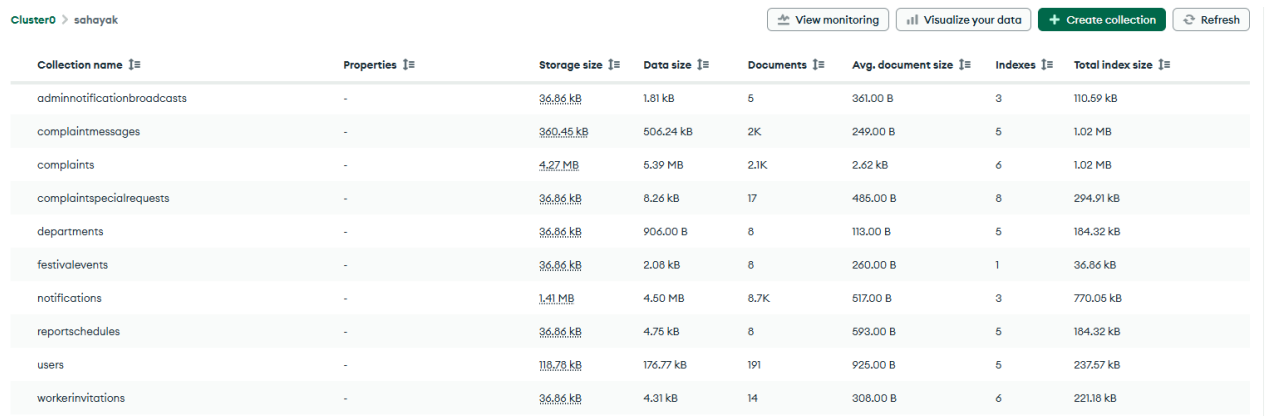
Logically, the main entities are User, Complaint, Notification, ReportSchedule, Invitation, and FestivalEvent. Complaint is the core operational entity and includes ticket identity, text, location, priority, department, status, workers, history, photos, feedback, and message threads. Users differ by role but share common identity and authentication attributes.

Entity	Important Fields
User	username, fullName, email, phone, role, department, notificationPreferences, preferredLanguage, rating, performanceMetrics
Complaint	ticketId, userId, rawText, refinedText, department, locationName, coordinates, priority, status, assignedWorkers, history, feedback, completionPhotos, proofImage, aiAnalysis, deleted
ComplaintMessage	complaintId, senderId, senderName, senderRole, text, createdAt
Notification	userId, title, body, type, data, readAt
Department	name, code, isActive
Invitation	email, department, role, invitedBy, tokenHash, expiresAt, acceptedAt, revokedAt
ReportSchedule	userId, email, frequency, format, cronExpression, department, timezone, isActive
ComplaintSpecialRequest	complaintId, requestType, currentDepartment, requestedDepartment, currentPriority, requestedPriority, requestedBy, status
AdminNotificationBroadcast	createdBy, title, body, audienceLabel, recipientRoles, recipientCount, deliveredCount, status

3.4.2 Physical Database Design

Physically, the system stores data as MongoDB collections with Mongoose schemas. Soft deletion is supported for complaints. Indexing and model-level defaults are used for important workflow fields such as ticket IDs, timeline history, and state metadata. The document-oriented approach reduces unnecessary join complexity and helps complaint detail queries deliver richer response payloads for the mobile app.

At the physical level, MongoDB collections are backed by Mongoose schemas and indexes. For example, complaints are indexed by userId, department, status, assigned worker, and creation time, while notifications are indexed by userId and read status. This indexing strategy supports efficient complaint listing, worker assignment retrieval, and notification history access.



Cluster0 > sahayak

View monitoring | Visualize your data | + Create collection | Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
adminnotificationbroadcasts	-	36.86 kB	1.81 kB	5	361.00 B	3	110.59 kB
complaintmessages	-	360.45 kB	506.24 kB	2K	249.00 B	5	1.02 MB
complaints	-	4.27 MB	5.39 MB	2.1K	2.62 kB	6	1.02 MB
complaintspecialrequests	-	36.86 kB	8.26 kB	17	485.00 B	8	294.91 kB
departments	-	36.86 kB	906.00 B	8	113.00 B	5	184.32 kB
festivalevents	-	36.86 kB	2.08 kB	8	260.00 B	1	36.86 kB
notifications	-	1.41 MB	4.50 MB	8.7K	517.00 B	3	770.05 kB
reportschedules	-	36.86 kB	4.75 kB	8	593.00 B	5	184.32 kB
users	-	118.78 kB	176.77 kB	191	925.00 B	5	237.57 kB
workerinvitations	-	36.86 kB	4.31 kB	14	308.00 B	6	221.18 kB

Img 3.4.2: MongoDB schema and collection structure used in the Sahayak backend.

Chapter 4. Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation

Category	Used In Project
Programming Languages	JavaScript, TypeScript (select utilities)
Backend Framework	Node.js, Express
Database	MongoDB with Mongoose
Mobile Framework	React Native with Expo and Expo Router
State Management	TanStack Query, local state hooks
Notifications	Expo Notifications
Image Support	Expo Image Picker, Cloudinary
Realtime	ws WebSocket
Reporting	ExcelJS, PDFKit, CSV generation
IDE / Editor	Visual Studio Code
Version Control	Git and GitHub
Media Storage	Cloudinary
Email Service	Resend (email delivery infrastructure)
AI Integration	Google Gemini-backed AI analysis and assistant services

These technologies were chosen because they support rapid full-stack development, cross-platform mobile delivery, flexible document-oriented storage, and modular backend architecture.

4.2 Testing Techniques and Test Plans (According to project)

The current project primarily uses practical validation and feature-level testing. Core testing areas include authentication flow, complaint submission validation, media upload, role-based authorization, assignment actions, worker status updates, feedback actions, and report generation. Since this is a mini project, the emphasis is on manual scenario testing, API path verification, seeded user-role checks, and regression checks during development.

Test Case	Expected Outcome
Citizen creates complaint with image and location	Complaint stored successfully and visible in citizen and HOD views
Worker updates status without authorization	Unauthorized transition blocked based on role/policy
HOD assigns complaint to worker	Assignment stored and worker queue updated
Worker uploads completion evidence	Complaint enters pending approval state
HOD approves complaint	Status changes to resolved and feedback flow becomes available
Admin restores deleted complaint	Complaint becomes visible in normal complaint flows again
Citizen attempts feedback before complaint resolution	Feedback should be blocked or unavailable until resolution stage
Worker submits pending approval without completion photos	System should reject the request
HOD marks complaint as needs-rework	Complaint should move to needs-rework and worker must be re-notified
Admin restores soft-deleted complaint	Complaint should reappear in standard complaint queries
HOD schedules report delivery	Schedule should be stored and visible in scheduled report listing

Testing in this project is mainly scenario-based and role-based. Since the application is built around actors and workflow transitions, test design focuses on whether the correct actor can perform the correct action in the correct complaint state. This includes validation of allowed transitions, blocked transitions, proper notification triggers, and successful persistence of uploaded evidence.

An important part of testing in Sahayak is workflow-state validation. The system enforces role-sensitive complaint state transitions through policy checks and workflow services. Therefore, test coverage should verify not only success cases but also invalid transitions, blocked role actions, and missing evidence conditions.

4.3 Installation Instructions

To install the backend, navigate to the backend directory and run the package installation command. Configure required environment variables such as MongoDB URI and JWT secret. Start the backend using the development script. For the mobile app, navigate to the mobile directory, install dependencies, and run the Expo start command. Ensure that the mobile app points to the correct backend API base URL. Seed data may be inserted using the backend seeding script for demonstration and workflow testing.

```
PS D:\Github\Sahayak> cd backend; npm install; npm run dev

added 1 package, and audited 284 packages in 3s

31 packages are looking for funding
  run `npm fund` for details

15 vulnerabilities (5 low, 1 moderate, 9 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.12.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.12.1
npm notice To update run: npm install -g npm@11.12.1
npm notice

> backend@0.0.0 dev
> nodemon ./bin/www

[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node ./bin/www`
[dotenv@17.2.1] injecting env (17) from .env -- tip: 🛠️ run anywhere with `dotenvx run -- yourcommand`
🔄 Self-ping cron job started (every 14 minutes)
Server starting on port 6000
MongoDB Connected
[event-priority] Cron job started (every 2 days at 02:00)
SLA escalation cron job started (hourly at minute 5)
[report-scheduler] Initialized 6 schedules
Backend server listening on port 6000
```

Figure 4.3.1: Terminal commands used for backend dependency installation, environment setup, and server execution.

4.4 End User Instructions

A citizen first registers or logs in, then opens the new complaint screen and fills in the issue description, evidence images, department selection, and location details. After submission, the citizen can track the complaint from the complaints list and detail view. A worker logs in to view assigned complaints, update work progress, and upload completion photos. A HOD logs in to review department complaints, assign workers, inspect AI review suggestions, and approve or rework submitted work. Admin-level users use dedicated flows for user and department management, deleted complaint recovery, special-request review, broadcast notification management, and festival-event configuration.

Chapter 5 Results and Discussions

5.1 User Interface Representation (of Respective Project)

The system provides separate role-based mobile interfaces. Citizen views emphasize ease of complaint reporting and tracking. Worker views emphasize work queue actionability and evidence upload. HOD views emphasize managerial oversight, assignment control, analytics, and reporting. These UI layers reflect the actual implementation found in the project.

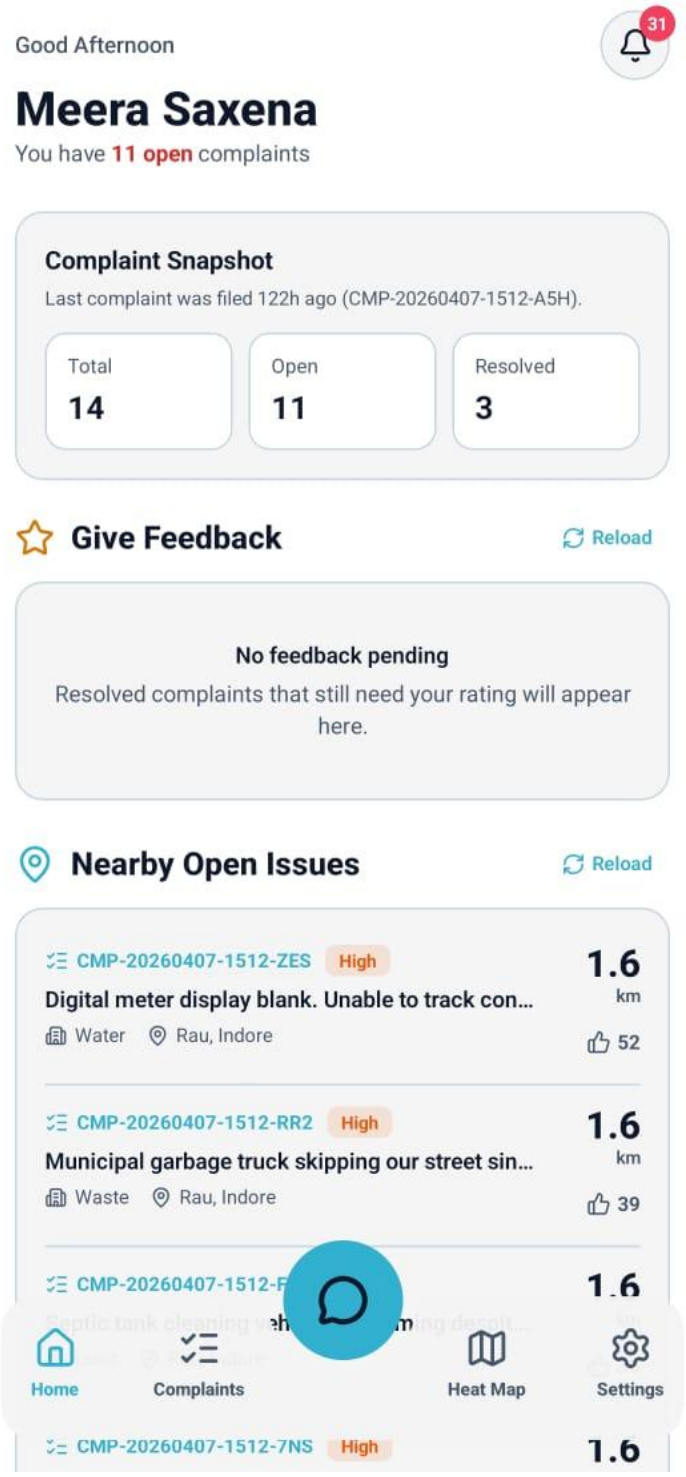


Figure 5.1.1: Citizen dashboard showing complaint overview, categories, and complaint access options.

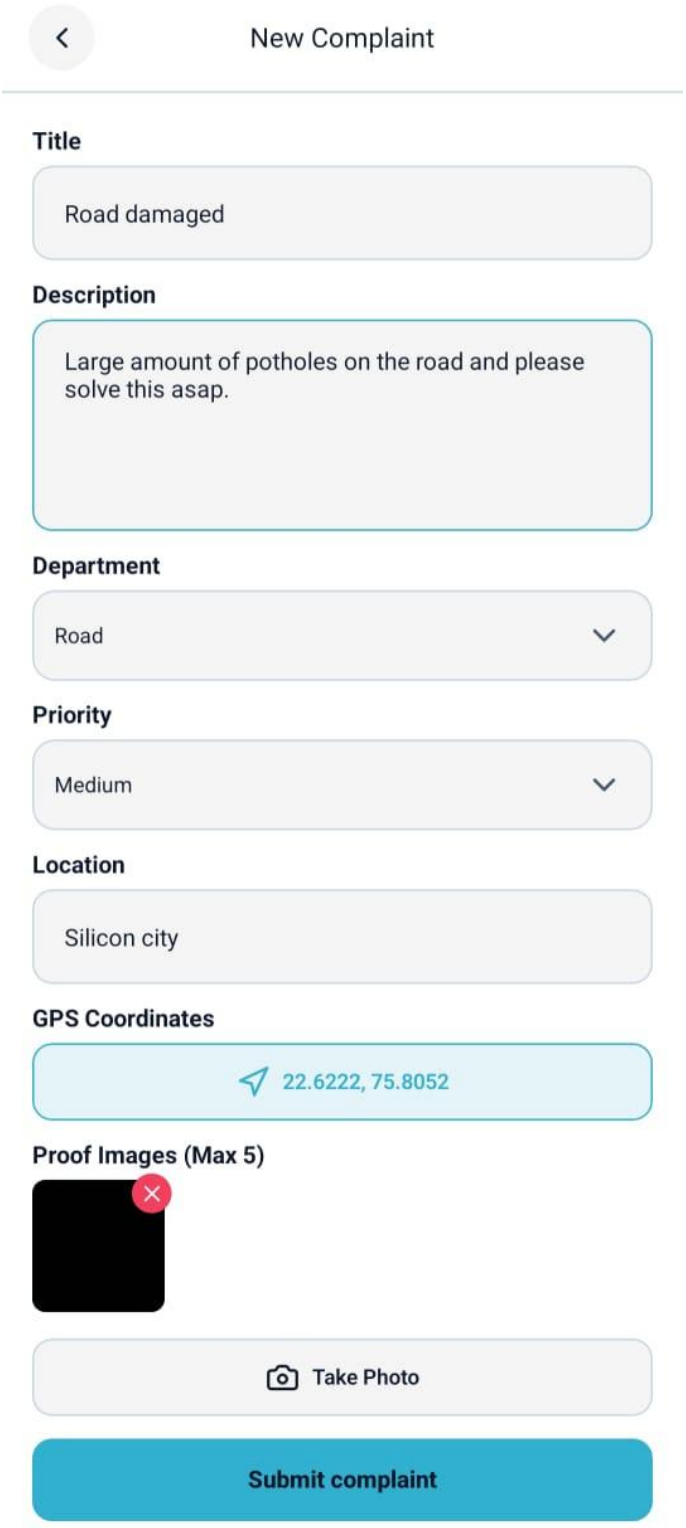


Figure 5.1.2: Complaint registration screen showing issue description, image evidence, and location capture.

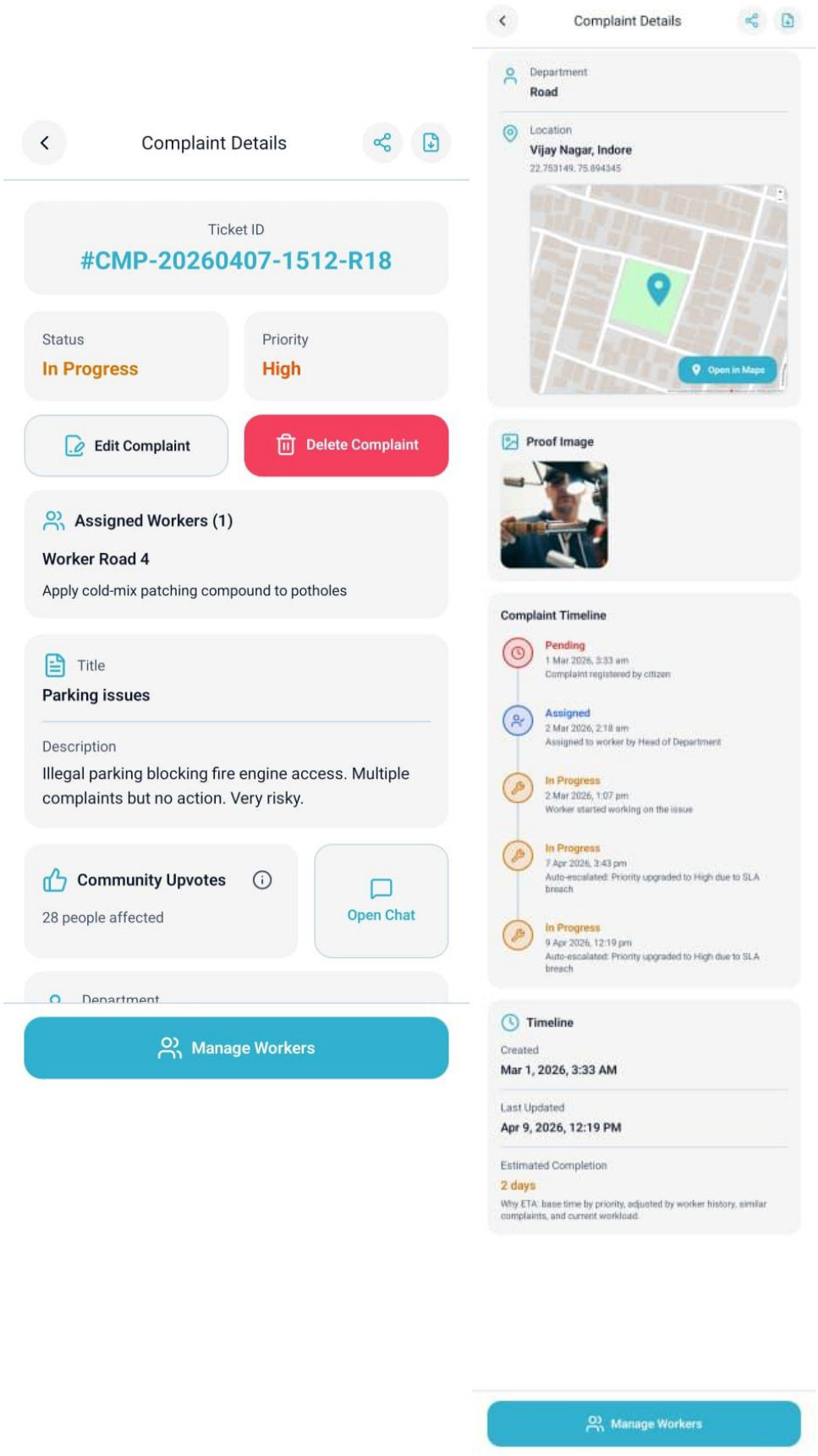


Figure 5.1.3: Complaint detail screen showing status, timeline, assigned workers, and complaint context.

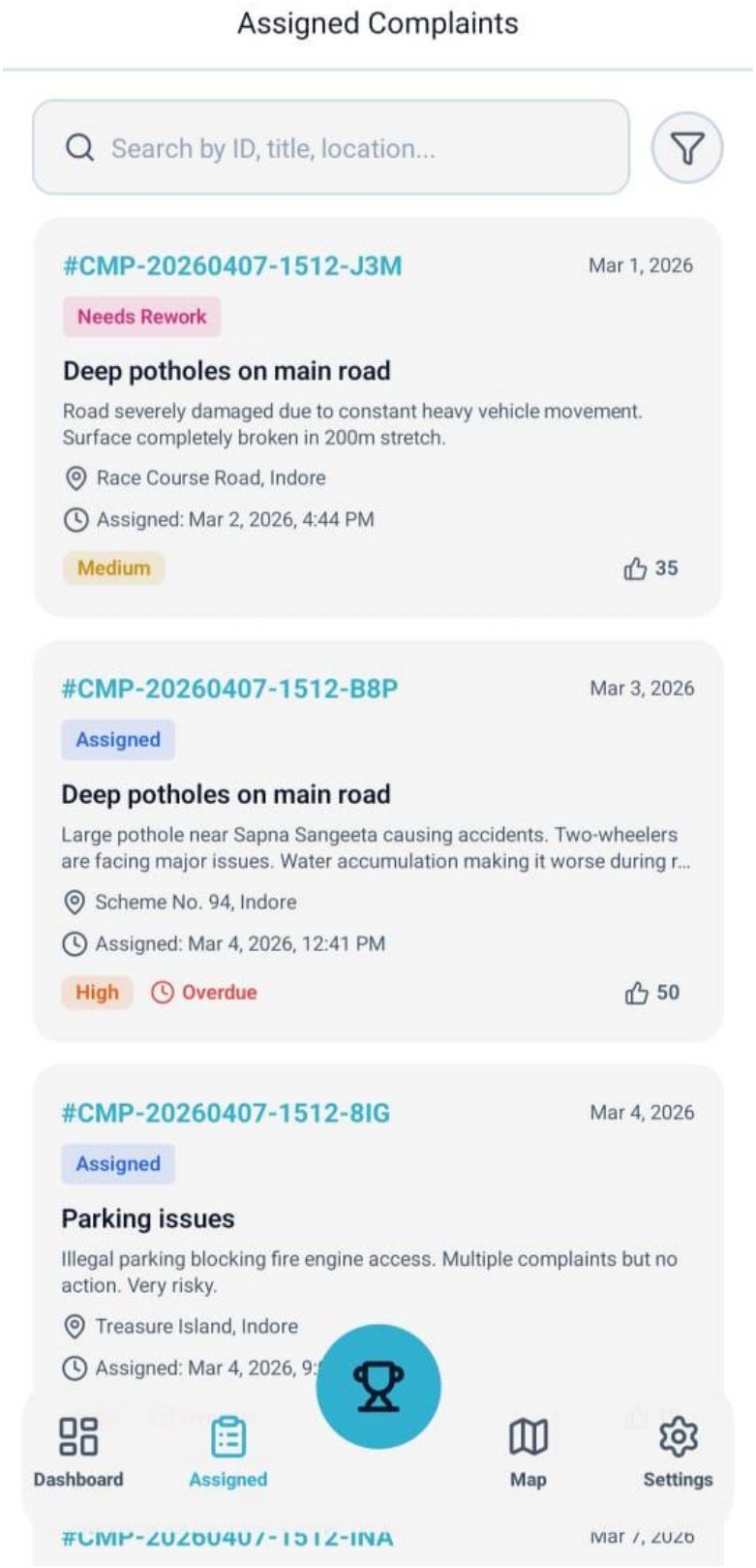


Figure 5.1.4: Worker assigned complaints screen showing actionable work queue and complaint status visibility.

5.2 Brief Description of Various Modules of the system

Module	Description
Authentication Module	Handles registration, login, refresh token flow, email verification, password reset, and profile access control
Complaint Module	Handles complaint creation, complaint detail retrieval, owner history, nearby complaints, upvotes, satisfaction votes, feedback, and complaint lifecycle display
Complaint Messaging Module	Handles complaint-specific message threads between citizen, worker, HOD, and admin actors
Worker Module	Handles assigned complaints, completed complaints, dashboard summary, leaderboard, analytics, feedback history, and status updates
HOD Module	Handles queue review, worker assignment, worker task editing, approval, rework, cancellation, invitation management, special requests, and reporting
Admin Module	Handles users, departments, deleted complaints, special-request review, broadcast notifications, and festival events
Notification Module	Handles push token registration, notification history, read-state updates, preferences, and admin broadcast history

Reporting Module	Handles PDF, Excel, CSV export, dashboard stats, department breakdown, email reports, and report scheduling
Assistant and AI Module	Handles AI-assisted complaint analysis, sentiment support, department suggestion, and assistant-driven conversational flows

5.3 Snapshots of system with brief detail of each

This section should include the final UI screenshots after visual selection. Each screenshot can be accompanied by a two to three sentence explanation of the feature shown. Typical screenshots should include citizen complaint creation, complaint detail, assistant/chat, worker assigned complaints, HOD worker assignment, HOD reports, notification preferences, and heatmap screen.

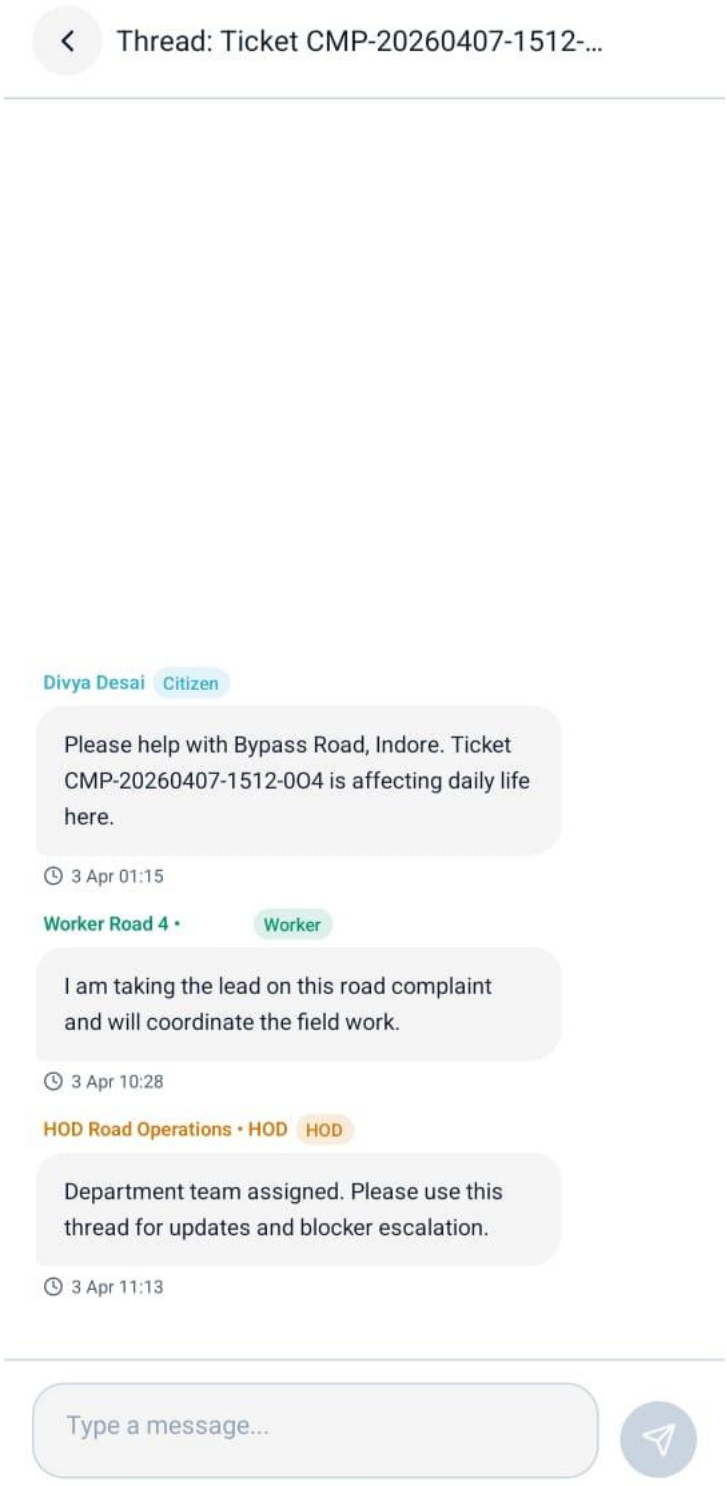


Figure 5.3.1: Complaint chat interface showing role-aware communication on complaint threads.

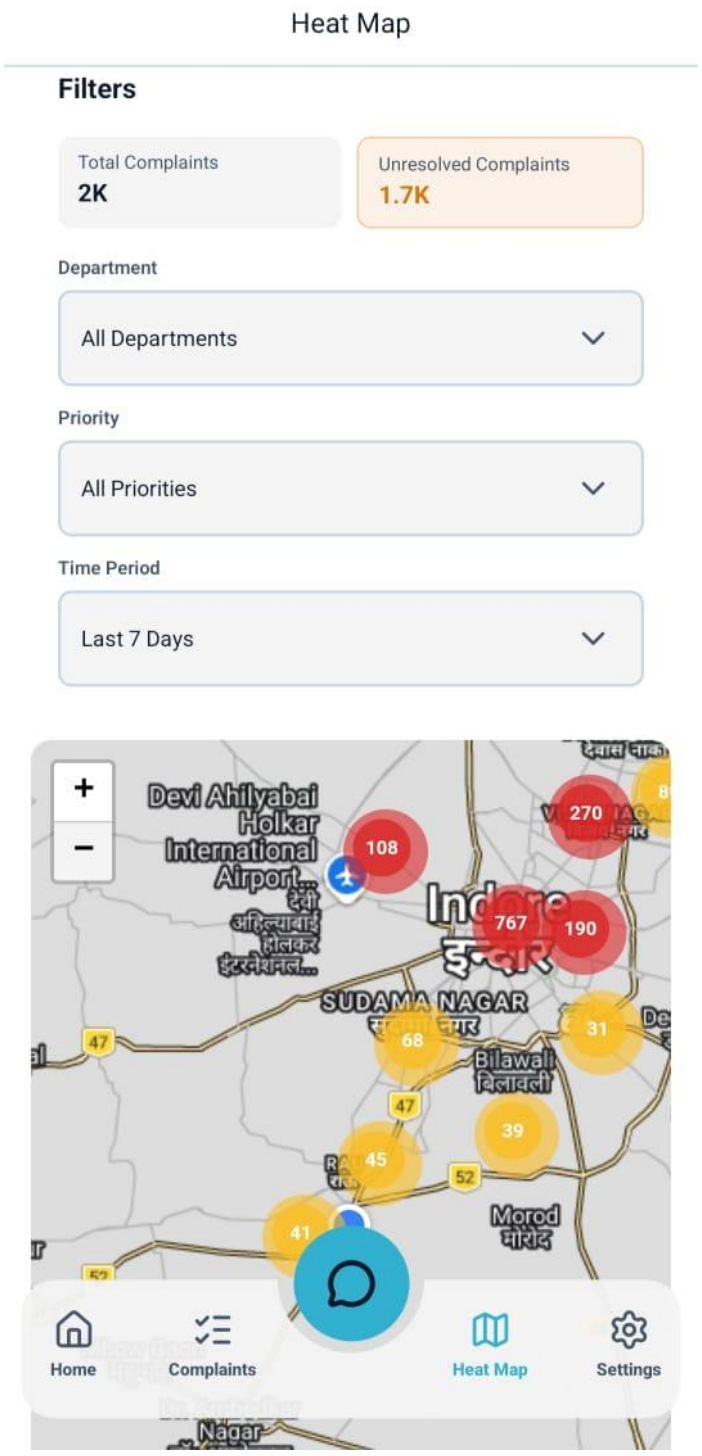


Figure 5.3.2: Heatmap interface showing complaint hotspot visualization based on complaint locations.

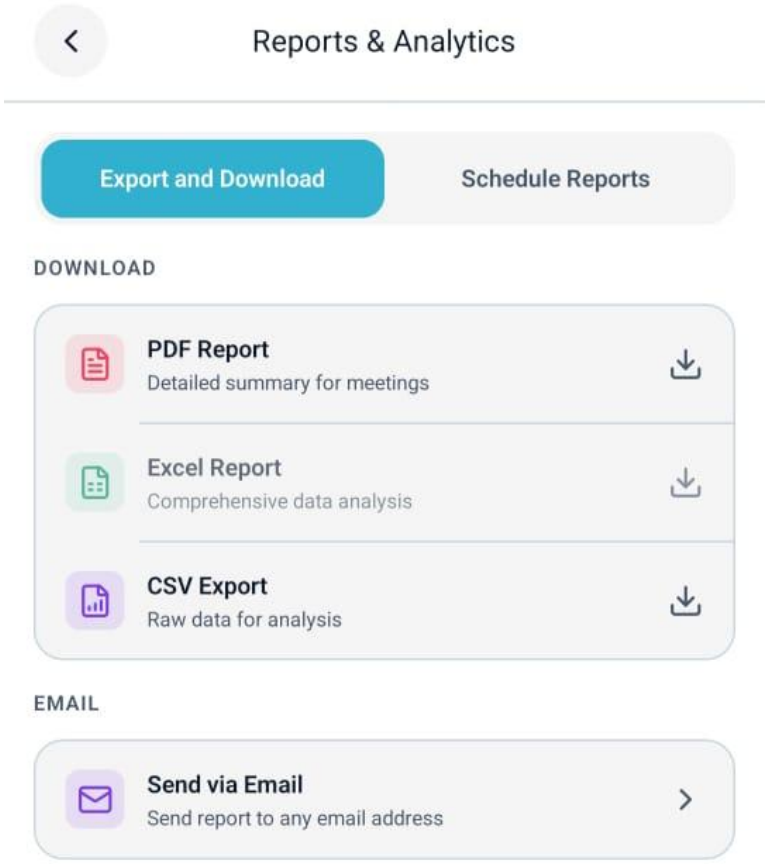
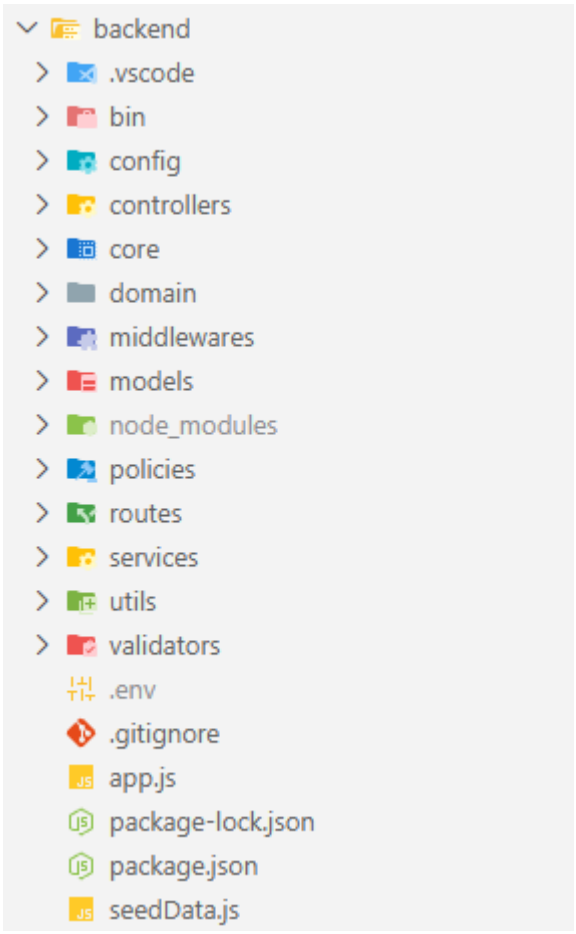


Figure 5.3.3: HOD report and schedule screen showing export and recurring reporting controls.

5.4 Back Ends Representation (Database to be used)

The backend is implemented using Node.js and Express, while the database used is MongoDB. Mongoose is used for schema modeling and access management. The backend is structured into routes, controllers, services, models, validators, middlewares, and utility modules. This separation helps maintain modularity and allows individual workflows to evolve without tightly coupling all code into a single file or layer.



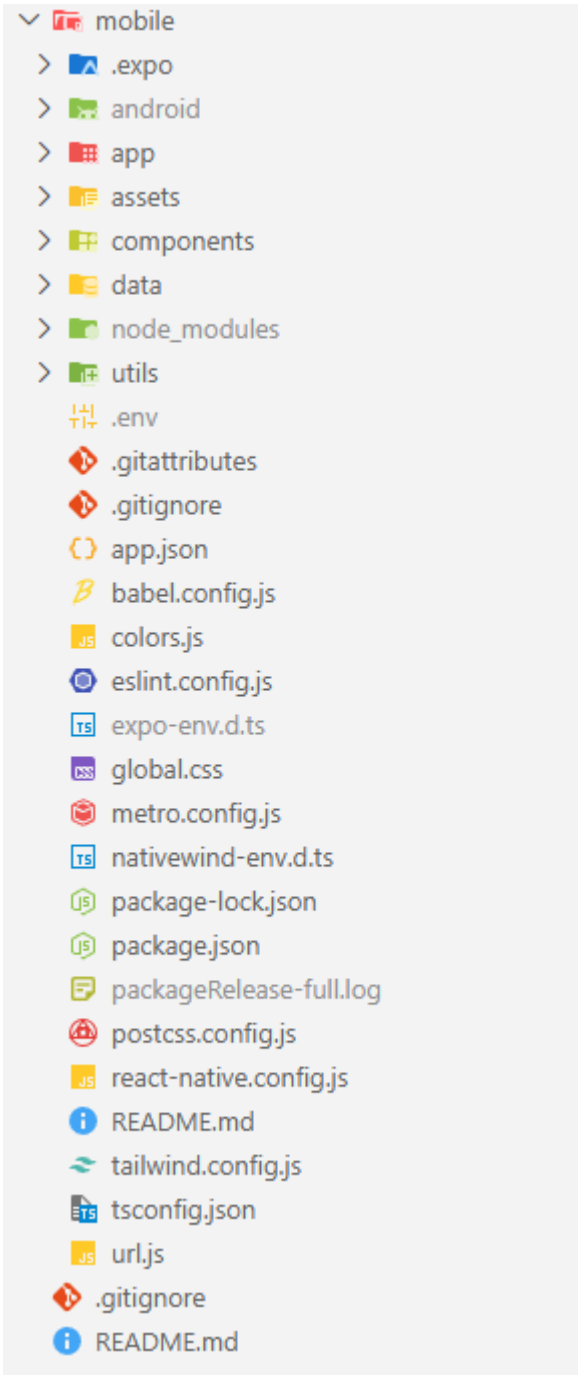
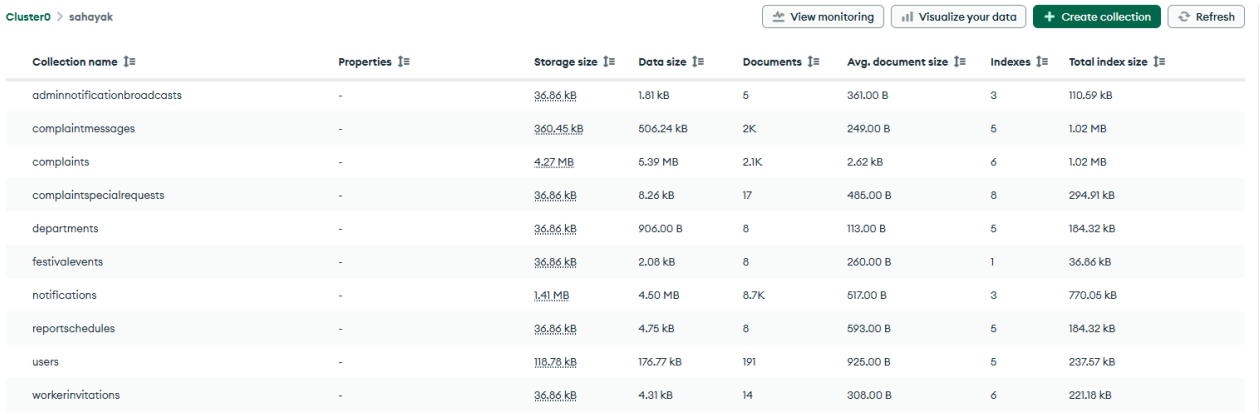


Figure 5.4.1: Backend folder structure and layered architecture of the Sahayak project.



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
adminnotificationbroadcasts	-	36.86 kB	1.81 kB	5	361.00 B	3	110.59 kB
complaintmessages	-	360.45 kB	506.24 kB	2K	249.00 B	5	1.02 MB
complaints	-	4.27 MB	5.39 MB	2.1K	2.62 kB	6	1.02 MB
complaintspecialrequests	-	36.86 kB	8.26 kB	17	485.00 B	8	294.91 kB
departments	-	36.86 kB	906.00 B	8	113.00 B	5	184.32 kB
festivalevents	-	36.86 kB	2.08 kB	8	260.00 B	1	36.86 kB
notifications	-	1.41 MB	4.50 MB	8.7K	517.00 B	3	770.05 kB
reportschedules	-	36.86 kB	4.75 kB	8	593.00 B	5	184.32 kB
users	-	118.78 kB	176.77 kB	191	925.00 B	5	237.57 kB
workerinvitations	-	36.86 kB	4.31 kB	14	308.00 B	6	221.18 kB

Figure 5.4.2: MongoDB collection view showing the main collections used by the system.

The backend architecture follows a route-controller-service-model pattern. Routes receive HTTP requests, controllers validate and orchestrate them, services implement domain-specific workflow logic, and models persist structured data through Mongoose schemas. This pattern is especially visible in complaint creation, assignment, notification, reporting, and workflow transition features.

5.5 Snapshots of Database Tables with brief description

Because the project uses MongoDB, this section should show collection snapshots or schema screenshots rather than relational tables. The most relevant examples are the User, Complaint, Notification, and ReportSchedule collections. For each screenshot, mention what the collection stores and why it is important in the workflow. Since MongoDB is document-oriented, embedded fields such as assignedWorkers, history, feedback, aiAnalysis, and satisfactionVotes should also be highlighted in the complaint collection screenshot description.

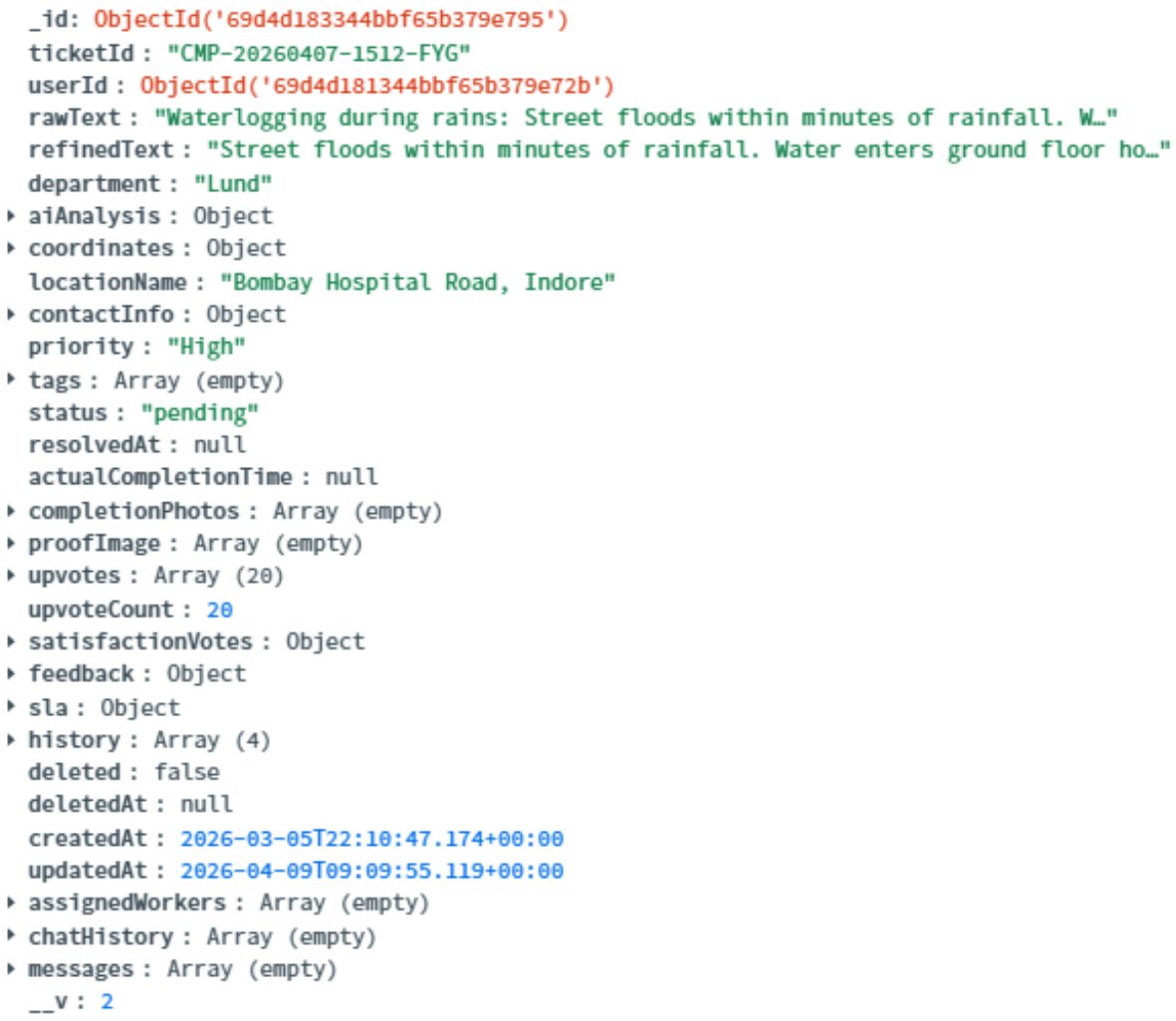


Figure 5.5.1: Complaint collection snapshot showing workflow state, department, assignments, and embedded history.

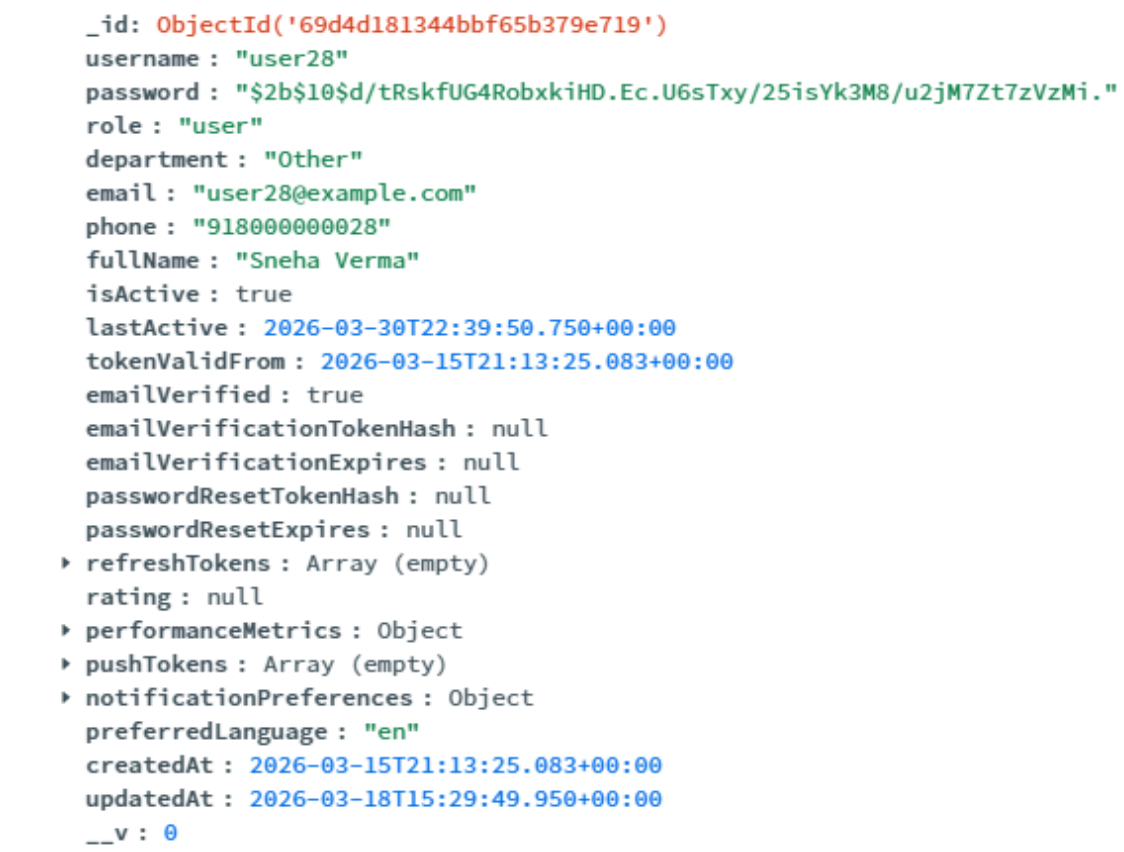


Figure 5.5.2: User collection snapshot showing role, department, preferences, and account metadata.

Chapter 6 Summary and Conclusions

Sahayak successfully demonstrates the design and implementation of a smart municipal complaint management system that supports multiple stakeholders and a full complaint lifecycle. The project transforms a common civic problem into a structured digital workflow supported by complaint registration, evidence capture, assignment, realtime communication, notifications, analytics, and reporting. It also shows that a student project can meaningfully combine frontend, backend, database, mobile, and workflow-driven engineering into one integrated application.

The final outcome is not merely a static interface but a role-aware platform that reflects realistic operational flows. The project contributes academically through applied software engineering practice and contributes practically by proposing a transparent and scalable model for municipal complaint handling.

The completed system reflects a realistic evolution from requirement study to deployable application design. It shows how modular backend design and role-aware mobile interfaces can be combined to build a civic-tech platform with practical relevance.

Chapter 7 Future scope

The future scope of Sahayak includes speech-based complaint capture directly from the mobile app, improved AI-assisted classification, deeper admin dashboards, public web transparency portals, map-based dispatch optimization, richer audit logs, automated test suites, and integration with official municipal information systems. The system can also be expanded into a broader smart-city governance platform by connecting civic assets, emergency workflows, and predictive issue clustering.

Future work may also include multilingual speech-to-text complaint submission, web dashboard support for departments, GPS-based workforce route optimization, public transparency portals for open civic issues, and stronger automated testing with performance monitoring.

Appendix

Appendix A. Repository Structure

Sahayak/
|- backend/
| |- app.js
| |- bin/www
| |- controllers/
| |- services/
| |- models/
| |- routes/
| |- middlewares/
| |- validators/
| |- utils/
| |- seedData.js
|- mobile/
| |- app/
| |- components/
| |- utils/
| |- assets/
| |- data/
| |- url.js
|- README.md

Appendix B. Important API Endpoints

Module	Endpoints
Auth	/api/auth/register, /api/auth/login, /api/auth/me, /api/auth/verify-email/:token, /api/auth/forgot-password
Complaints	/api/complaints, /api/complaints/nearby, /api/complaints/:complaintId, /api/complaints/:id/messages, /api/complaints/:complaintId/feedback
Worker	/api/workers/dashboard-summary, /api/workers/assigned-complaints, /api/workers/completed-complaints, /api/workers/complaint/:complaintId/status
HOD	/api/hod/overview, /api/hod/workers, /api/hod/complaints/:complaintId/assign-workers, /api/hod/approve-completion/:complaintId, /api/hod/needs-rework/:complaintId
Reports	/api/reports/pdf, /api/reports/excel, /api/reports/csv, /api/reports/schedule
Notifications	/api/notifications/history, /api/notifications/preferences, /api/notifications/admin-broadcasts
Chat / AI	/api/chat/message, /api/chat/speech-to-text
Admin	/api/users, /api/departments, /api/complaints/deleted, /api/complaints/:complaintId/restore

Appendix C. Sample Test Accounts

Role	Username	Password
Admin	admin1	password123
HOD	hod_road	password123
Worker	worker_road_1	password123
Citizen	user1	password123

These seeded credentials are intended for local testing and demonstration of the workflow defined in the project repository.

Bibliography

[1] Project repository documentation, "Sahayak README," local project documentation, 2026.

[2] Project repository documentation, "Sahayak FUTURE.md," local audit and roadmap notes, 2026.

[3] Node.js Foundation, "Node.js Documentation," official documentation.

[4] Express.js, "Express Web Framework Documentation," official documentation.

[5] MongoDB Inc., "MongoDB Manual," official documentation.

[6] Mongoose Team, "Mongoose ODM Documentation," official documentation.

[7] Expo, "Expo and Expo Router Documentation," official documentation.

[8] Meta Platforms, "React Native Documentation," official documentation.

[9] TanStack, "TanStack Query Documentation," official documentation.

[10] Cloudinary, "Cloudinary Documentation," official documentation.

[11] WebSocket API package authors, "ws Documentation," official package documentation.

[12] node-cron authors, "node-cron Documentation," official package documentation.

[13] Google, "Gemini API / Gemini Documentation," official documentation.

[14] Resend, "Resend Email API Documentation," official documentation.

List of Publications

No publications are associated with this minor project at the time of submission.